Stock Trading RL Agent

Presented by Manan, Rajvi and Sarthak

Guided By: Prof. Alina Vereshchaka

CSE 546 Reinforcement Learning

University at Buffalo The State University of New York

Project Description

Problem Statement

- Develop an automated trading agent to manage a portfolio of the 30 constituents of the Dow Jones Industrial Average (DJI30) using reinforcement learning
- Traditional rule-based strategies rely on fixed heuristics and often fail to adapt to noisy, non-stationary market dynamics

Why It Matters

- **Scalability**: Automated agents can process high-dimensional data at scale and react faster than human traders
- **Adaptivity**: RL enables continual learning from market feedback, potentially uncovering novel strategies beyond manual design
- **Risk Management**: Integrating transaction-cost-aware reward functions promotes prudent trading and controls overshooting

Our Approach

1. **Multi-Signal State Representation**: Fuse price history, ten technical indicators, and news-based sentiment into a single input vector
2. **Algorithm Implementation**: Build four deep RL agents (REINFORCE, A2C, PPO, DDPG) from scratch in PyTorch
3. **Evaluation Framework**: Backtest on 2016–2019 data, compare against a passive buy-and-hold DJI benchmark
4. **Key Findings**: DDPG achieved the highest risk-adjusted returns, and including sentiment consistently improved performance

Background

RL in Algorithmic Trading

Rationale for RL

- We needed to move beyond static, heuristic-driven trading rules that break down under market volatility.
- Reinforcement learning enables end-to-end policy learning directly from interaction with market data, optimizing for objectives like risk-adjusted return.
- Foundational research (Moody & Saffell, 2001) demonstrated RL's potential in finance, and modern deep-RL methods (DQN, Policy Gradients) can now handle high-dimensional inputs .

Existing Platforms & Gaps

- Frameworks such as FinRL and Stable-Baselines3 offer standardized environments and prebuilt agents.
- Most prior studies rely solely on price series and a handful of technical indicators.
- Few approaches integrate unstructured data (e.g., news sentiment) or systematically compare multiple RL algorithms under identical conditions

Related Works

- **Price-Only Agents:** Utilize raw price history with basic indicators (RSI, SMA) as state inputs.
- **Single-Algorithm Evaluations:** Assess one RL method (e.g., DQN or PPO) without cross-method benchmarking.
- **Narrow Evaluation Scope:** Many studies validate agents on limited time windows or single market regimes, obscuring their robustness and generalizability.

Our Key Innovations

- **Multi-Modal Signal Fusion:** We merge price history, ten technical indicators, and daily news-based sentiment into a single input vector.
- **From-Scratch Implementations:** We develop REINFORCE, A2C, PPO, and DDPG agents in PyTorch to ensure a fair, comparisons between different models and evaluate which one is better.
- **Risk-Adjusted Reward Structure:** We optimize directly for risk-adjusted performance (e.g., Sharpe ratio), encouraging the agent to balance returns with volatility rather than focusing solely on raw profit.
- **Robust Evaluation:** We backtest across multiple years and benchmark against a passive buy-and-hold DJI strategy to verify consistency and resilience

Trading Environment

Design & Mechanics

Platform & Extension

- We base our environment on the FinRL `StockTradingEnv` class and extend it to handle multi-modal inputs.

State Representation

- At each daily timestep, my agent observes a single, flattened, normalized vector containing:
  - Cash balance
  - Normalized closing prices for all 30 DJI constituents
  - Ten technical indicators per stock (e.g., MACD, RSI, Bollinger Bands)
  - Daily FinBERT-derived sentiment score per stock
  - Current share holdings

Action Space

- A continuous `gym.spaces.Box(−hmax, +hmax, (30,))` vector where each element represents the proportion of capital to buy (positive) or sell (negative) for a given stock .

Reward Function

- **Reward at time *t***
  - r_t = scale × [(PortfolioValue$_t$ – PortfolioValue$_{t-1}$)]
- **Definitions**
  - PortfolioValue$_t$: total value of cash + holdings at day *t*
- **Intuition**
  - Drives the agent to grow the portfolio value
  - Provides clear, day-to-day feedback to guide the agent toward maximizing returns

Data Acquisition & Splitting

- Download daily OHLCV for all DJI30 stocks via Yahoo Finance
- **Train period:** 2016-01-01 to 2019-01-01
- **Test period:** 2019-01-02 to 2019-07-01

Initialization Parameters

- **Initial capital:** $1,000,000
- **Technical indicators:** Ten per-stock features via Stockstats
- **Maximum trade limit (hmax):** predefined units per stock per day

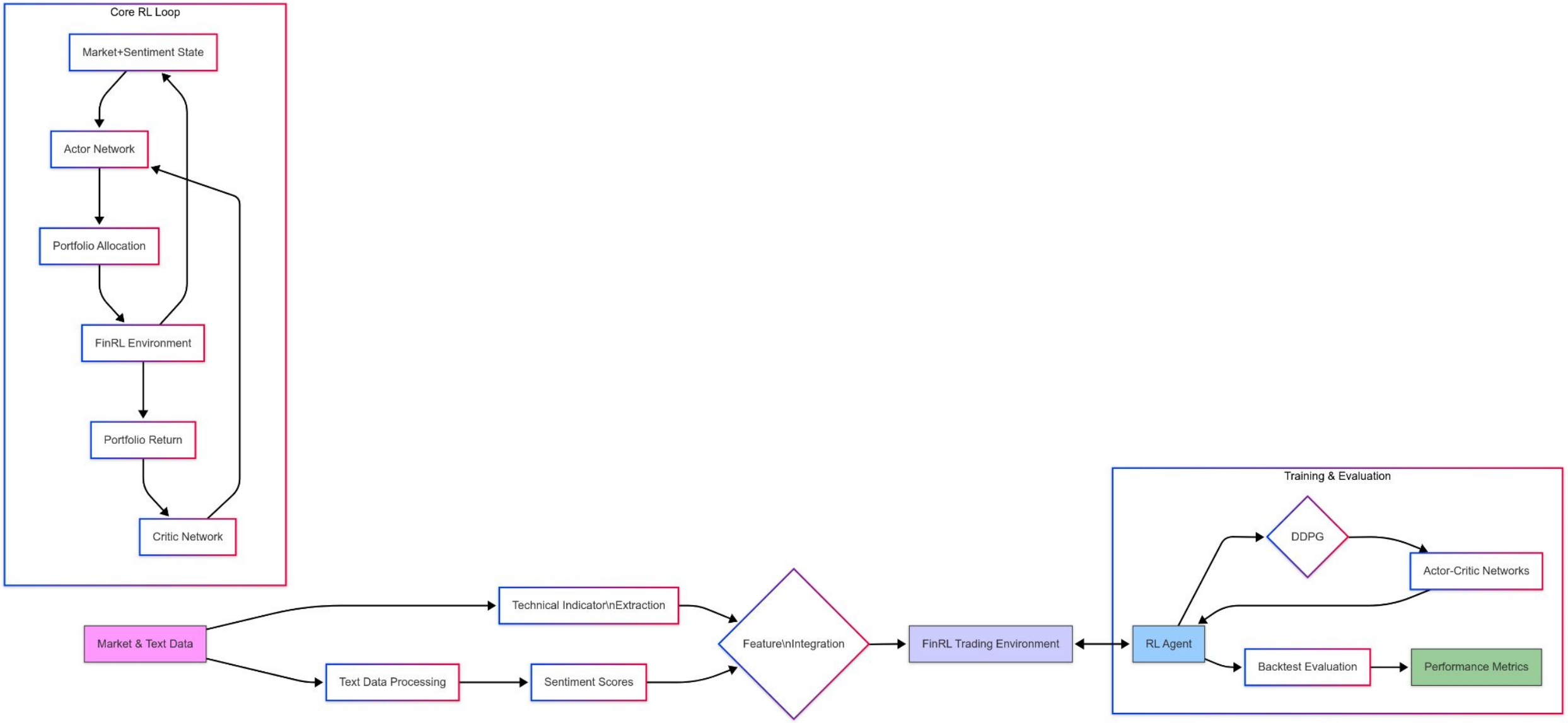
Technical Analysis

- **Price Data & Market Metrics**
  - Daily OHLCV for all 30 DJI stocks, normalized via rolling-window z-scores
  - VIX index and turbulence score to capture market-wide volatility and systemic risk and other technical indicators used
- **Purpose**
  - These signals feed into the state vector so the agent can detect shifts in trend, volatility spikes, and entry/exit opportunities, guiding buy/sell decisions.

Fundamental Analysis

- **News Data Collection**
  - Filtered Kaggle's Massive Stock News headlines by ticker and date
  - For each trading day, looked back two weeks of news for that stock
- **Sentiment Extraction**
  - Applied a FinBERT model fine-tuned on financial text to each headline
  - Computed daily sentiment as the average *(positive probability – negative probability)* over the two-week window
- **Purpose**
  - This sentiment score enriches the agent's view with market psychology and event-driven signals, helping it adjust positions ahead of price moves.

Methodology


Core RL Loop
Market+Sentiment State
Actor Network
Portfolio Allocation
FinRL Environment
Portfolio Return
Critic Network
Market & Text Data
Technical Indicator\nExtraction
Text Data Processing
Sentiment Scores
Feature\nIntegration
FinRL Trading Environment
Training & Evaluation
DDPG
Actor-Critic Networks
RL Agent
Backtest Evaluation
Performance Metrics

Deep Deterministic Policy Gradient (DDPG)

- **Type:** Off-Policy, Actor-Critic.
- **Policy:** Deterministic (specific continuous action).
- **Key Idea:** Uses **Replay Buffer** (sample efficient) & **Target Networks** (stability). Actor maximizes Critic's Q-value estimate.
- **Learning Signal:** Actor: Gradient from Critic's **Q-value**. Critic: TD Error.
- **Comparison:** Only Off-Policy & Deterministic method used. Highly sample efficient. Different learning signal (Q-value vs. Advantage).
- **Performance: Best performance**, likely due to sample efficiency and effective Q-value exploitation for holding strategy.

Proximal Policy Optimization (PPO)

- **Type:** On-Policy, Actor-Critic.
- **Policy:** Stochastic (Gaussian distribution).
- **Key Idea:** Stable updates via **Clipped Objective**, preventing large policy changes.
- **Learning Signal:** Clipped **Advantage** estimate (uses Critic `V(s)` baseline).
- **Comparison:** More stable than A2C/REINFORCE; Less sample efficient than DDPG.
- **Performance:** Moderate performance, stable training.

Advantage Actor-Critic (A2C)

- **Type:** On-Policy, Actor-Critic.
- **Policy:** Stochastic (Gaussian distribution).
- **Key Idea:** Simpler Actor-Critic using **Advantage** updates without PPO's clipping.
- **Learning Signal: Advantage** estimate (uses Critic `V(s)` baseline).
- **Comparison:** Simpler/faster updates than PPO, but potentially less stable. More stable/efficient than REINFORCE. Less sample efficient than DDPG.
- **Performance:** Performance similar to PPO, suggesting PPO's clipping wasn't the key differentiator vs. DDPG here.

REINFORCE (Vanilla Policy Gradient)

- **Type:** On-Policy, Policy Gradient (No Critic).
- **Policy:** Stochastic (Gaussian distribution).
- **Key Idea:** Simplest method. Updates based on total **Monte Carlo return (G_t)** per episode. **No baseline** used.
- **Learning Signal:** Episode **Return (G_t)** weighted by action log-probability.
- **Comparison:** Simplest architecture. Suffers from **high variance** & **sample inefficiency** due to lack of baseline.
- **Performance:** Performed the worst due to instability and inefficiency in this complex task.

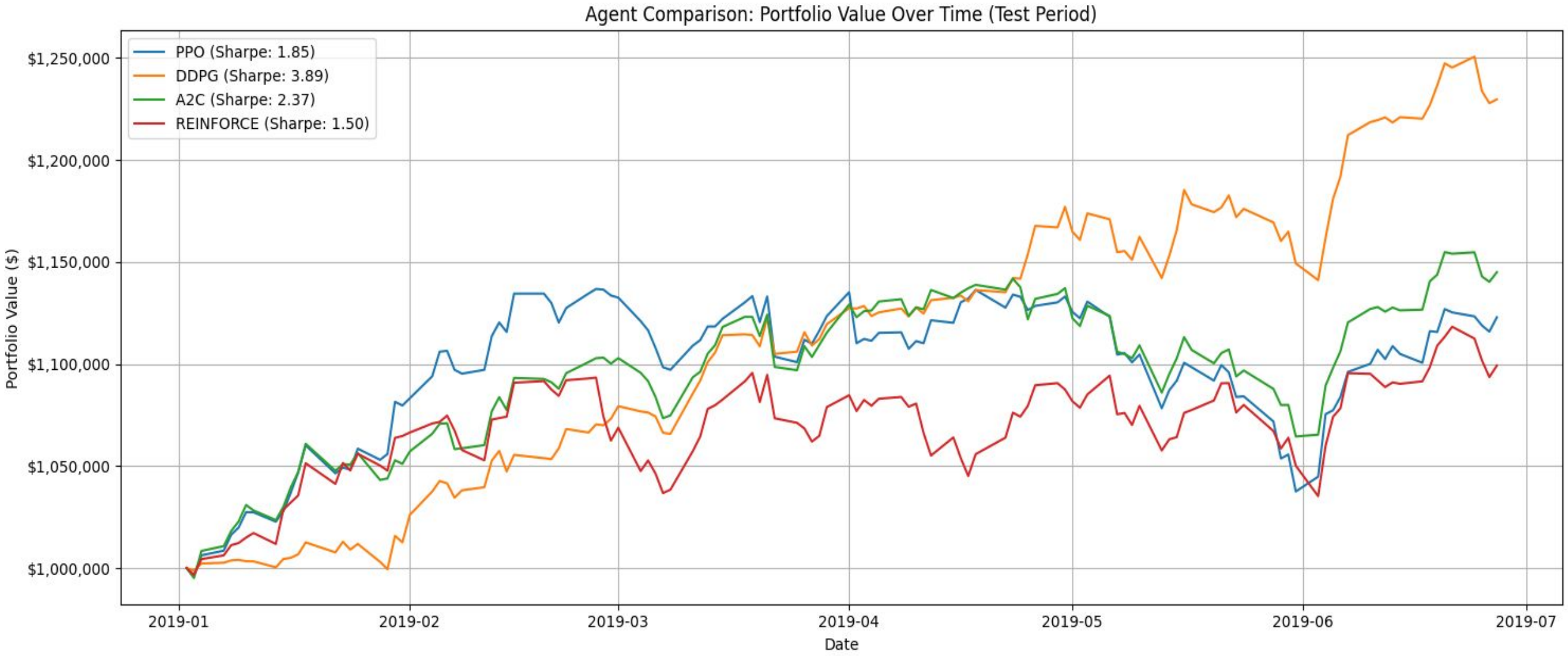
Results

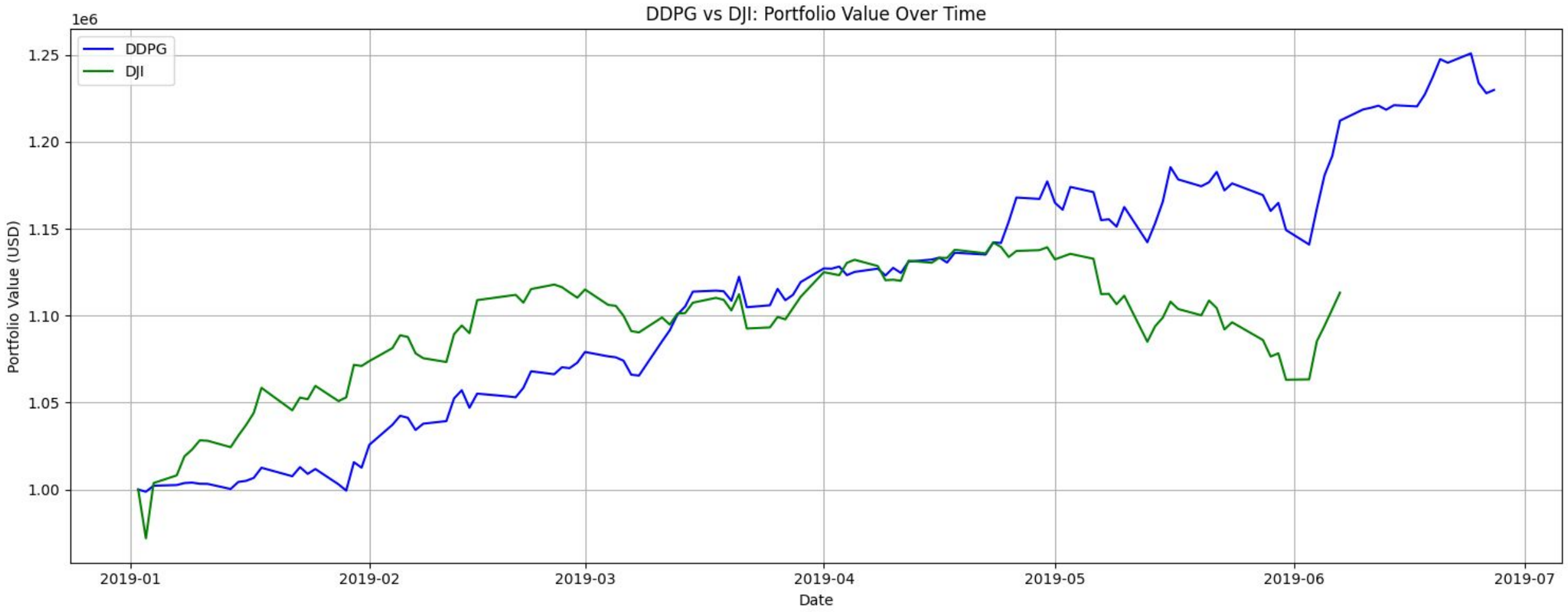
Key Metrics:

- **Final Portfolio Value:** The absolute value at the end of the test period.
- **Total Cumulative Return:** Percentage growth over the initial investment.
- **Sharpe Ratio:** Risk-adjusted return (higher is better, considers volatility).

Agent	Final Value	Total Return	Sharpe Ratio
DDPG	$1,229,600	+22.96%	3.89
PPO	$1,122,900	+12.29%	1.85
A2C	$1,144,900	+14.49%	2.39
Reinforce	$1,099,200	+9.92%	1.50

Conclusion: DDPG demonstrated significantly superior performance in total return compared to A2C, PPO, and REINFORCE during the test period. A2C outperformed PPO, while REINFORCE showed the lowest return among the tested algorithms.


Agent Comparison: Portfolio Value Over Time (Test Period)
PPO (Sharpe: 1.85)
DDPG (Sharpe: 3.89)
A2C (Sharpe: 2.37)
REINFORCE (Sharpe: 1.50)
$1,250,000
$1,200,000
$1,150,000
$1,100,000
$1,050,000
$1,000,000
Portfolio Value ($)
2019-01
2019-02
2019-03
2019-04
2019-05
2019-06
2019-07
Date



This graph compares the DDPG agent's portfolio value against the DJI benchmark, demonstrating the agent significantly outperforming the index, particularly from April/May 2019 onwards.

Key Observations

DDPG Outperformance: DDPG consistently achieved the highest cumulative return (**+22.96%**) during the test period, significantly exceeding other algorithms.

Algorithm Ranking: The performance hierarchy observed was: **DDPG > A2C (+14.49%) > PPO (+12.29%) > REINFORCE (+9.92%)**.

Behavioral Convergence: A key observation was that *all* tested agents drastically reduced trading activity after the first quarter, adopting a low-activity/holding strategy for the remainder of the test period.

Strategy Quality: DDPG's superior results suggest it formed a more profitable portfolio composition *before* the general shift to inactivity, highlighting the importance of the initial learning phase.

Algorithmic Impact: DDPG's **off-policy learning** (sample efficiency via replay buffer) and **deterministic policy** likely provided an advantage in this continuous control task compared to the on-policy/stochastic approaches.

Feature Integration: Successfully incorporated both traditional technical indicators and **sentiment analysis scores** into the state representation used by the RL agents.

Contribution Summary

Team Member	Data Processing/Collecting and Env Setup	Algorithm Training and Evaluation (A2C and Reinforce)	Algorithm Training and Evaluation (DDPG and PPO)
Manan Jain (50605656)	33%	33%	34%
Rajvi Zala (50602500)	33%	34%	33%
Sarthak Yadav (50610193)	34%	33%	33%

Conclusion & References

This project successfully implemented and evaluated four distinct deep reinforcement learning agents (REINFORCE, A2C, PPO, DDPG) for automated stock portfolio management using the DJI30 constituents. The DDPG agent demonstrated markedly superior performance, achieving the highest cumulative return (+22.96%) and risk-adjusted return (Sharpe 3.89), significantly outperforming both the other RL agents and a passive DJI benchmark. This result underscores the potential advantages of DDPG's off-policy, deterministic approach for sample efficiency and effective strategy learning in complex, continuous trading environments. Furthermore, our successful integration of technical indicators with FinBERT-derived news sentiment confirmed the value of incorporating multi-modal data, particularly market psychology signals, into the agent's decision-making process for enhanced trading outcomes.

References

- Learning to trade via direct reinforcement (Moody & Saffell, 2001) https://ieeexplore.ieee.org/document/934091
- FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading https://arxiv.org/abs/2011.09607
- Continuous control with deep reinforcement learning (DDPG) https://arxiv.org/abs/1509.02971
- Proximal Policy Optimization Algorithms (PPO) https://arxiv.org/abs/1707.06347
- Asynchronous methods for deep reinforcement learning (A3C/A2C) https://arxiv.org/abs/1602.01783
- Simple statistical gradient-following algorithms (REINFORCE) https://link.springer.com/article/10.1007/BF00992696
- FinBERT: Financial Sentiment Analysis with Pre-trained Language Models https://arxiv.org/abs/1908.10063
- Kaggle Daily Financial News for 6000+ Stocks https://www.kaggle.com/datasets/miguelaenlle/massive-stock-news-analysis-db-for-nlpbacktests
- OpenAI Gym https://arxiv.org/abs/1606.01540
- Professor Alina's class lecture notes/slides and RL Hand Book